

Does a Music Transformer Know What Key It Is In?

Causal Interventions on an Emergent Tonal Representation

Anonymous ICASSP 2027 submission

Abstract—We can decode musical attributes from the activations of a music generation model, but decoding does not show that the model uses what we decode. We test use directly. We train Transformers on symbolic music whose vocabulary contains only absolute pitch and time, and we replace the low-dimensional key component of the hidden state with the component of a different key while we keep the musical prompt fixed. On a held-out test set, with the layer, the subspace, the metrics and the statistical tests all fixed in advance, the edit moves generation to the injected key $9.1\times$ as often as a rank-matched random edit does (0.356 against 0.039; all 12 keys separate after Holm correction), and it also beats a control that we rescale to the same perturbation magnitude. The effect appears before any generated note can feed back: without sampling a single new token, the edit shifts the next-pitch distribution toward the injected key (+0.26 probability mass, against +0.02 for the random control, in 12/12 keys after Holm correction). We also find two dissociations. The key is decodable across many layers, but the edit changes the output only in a narrow middle band, and an edit restricted to pitch positions does nothing at all. The effect is therefore not a direct write to pitch decisions, and linear decodability is a weak guide to causal use. Finally, both the probe and the edit transfer to a public model trained on real music ($7.8\times$ the matched control, 12/12 keys). We conclude that a music Transformer computes a manipulable key representation and consults it during generation.

Index Terms—music generation, interpretability, causal intervention, tonality, probing

I. INTRODUCTION

Music generation models write convincingly tonal pieces [1], [2], [3]. They settle into a home key, move away from it, and return. Recent work shows that we can read musical attributes, including key, out of the activations of such models with trained classifiers [4]. Reading an attribute, however, does not show that the model uses it. A signal can be decodable from a representation while the model ignores it when it acts [5], [6]. Nobody has yet tested whether a music model uses an internal key representation when it generates.

Other domains already apply the causal test. A model trained only to predict Othello moves builds an internal

board, and editing that board changes which moves the model treats as legal [7], [8]. Researchers have applied the same standard to chess, mazes, and reinforcement learning [9], [10], [11].

Music has probes, and it has steering. Steering methods adapt inference-time interventions from language models [12], [13] and push an attribute along a learned direction [14], [15]. Such a result shows that a direction influences an attribute. It does not show that the model holds a representation whose *value* we can set: remove the current value, install a different one, and observe the model act on the new value.

Recent studies have directly compared direction-finding methods in language models. Wu et al. [16] introduced AxBench and compared several methods, including difference-of-means, linear probes, and sparse autoencoders (SAEs). Their results showed that simple baselines such as difference-of-means often achieved stronger steering performance than SAE-based methods. Arad et al. [17] further showed that the performance of SAE-based steering strongly depends on how one selects SAE features, and that selecting features by their measured effect on model outputs can outperform selection based only on activation statistics. These findings suggest that a direction associated with a target attribute is not necessarily the direction that most effectively controls that attribute. We extend this observation from language to music, and from feature selection to network depth and token position.

We run the causal test with musical key as the state variable, for three reasons. First, theory and psychological measurement define key outside the model [18], so the state we probe does not depend on our own labeling. Second, the model must compute key by integrating pitch evidence over time, because the vocabulary names no key. Third, the scale of a key gives us an objective success measure, so we do not need a learned judge. We replace the key component of one layer’s activations with the component of a donor key, and we ask what the model composes next. We fix every analysis choice before we look at the confirmatory data, we add a logit-level test that removes feedback from generated notes, and we add token-type ablations that rule out a direct write to pitch decisions.

II. METHOD

A. Data, models, and protocol

We need the true key at every note, so we generate the training data. **D-SYN** comes from a functional-harmony state machine that voices each chord for four

parts and optionally adds a melody. It writes 200k/10k/10k train/validation/test pieces of 278–508 tokens, and each piece contains 0–3 modulations. The machine knows the key it writes in, so every token carries a true key label $\kappa(t)$ over 24 classes. The machine draws keys near-uniformly: in the test split, every major key occupies between 214k and 239k tokens (max/min 1.12), so no key is rare. **We never give these labels to the model:** they train and score the probe only. The vocabulary holds 124 types—BAR, POS, absolute-MIDI PITCH, DUR, and structural symbols—and a unit test enforces the absence of key, chord, and degree tokens.

M-CTRL is a GPT-2 style decoder [19], [20] with 8 layers, $d=512$, and about 25M parameters. We train it with and without transposition augmentation, with three seeds each, which gives six models. **M-REF** shares the architecture but uses a different seed and a disjoint slice of the data. We never probe or edit M-REF; it only judges whether edited music still sounds like music, so no model grades its own work. All six models clear a quality gate that we set in advance: validation next-token top-1 of 0.878–0.881, against 0.434 for a predictor that always emits the most frequent token. We fixed all decision rules and controls before running, log every deviation, and report negative results.

B. Probing with controls

We train linear probes [21], [6] that map the residual stream $h_\ell(t)$ to $\kappa(t)$, and we split train and test by piece. We bound the probes with three control families. **C1** permutes the key labels within each piece; the score of that probe on the true keys gives the *selectivity floor* [5], which we subtract from every margin we report. **C2** is an untrained model. **C3** names the key from the notes alone: logistic regression on a pitch-class histogram, and Krumhansl–Schmuckler (KS) template matching [22], [23], over windows $W \in \{8, \dots, 64\}$. Section III-A extends C3 to the full prefix. Our frozen rule **DR-H1** requires the selectivity-corrected probe F_1 to beat the best C3 baseline with a piece-level BCa [24] 95% confidence interval that excludes zero.

C. Intervention

We prompt the model with 8 bars in a stable key. From bar 9 on, we edit one layer’s activations at every generated position, following the activation patching literature [25], [26]:

$$h \leftarrow h - P_V h + P_V \mu_{\kappa^*}, \quad (1)$$

where V is an orthonormal key subspace, $P_V = VV^\top$ projects onto it, and μ_{κ^*} is the mean activation of the donor key κ^* . **We never change the prompt.** We build V in three ways: the probe row space (**V-PROBE**), the span of the per-key means (**V-MEAN**), and an optimized subspace trained by distributed alignment search (**V-DAS**) [27].

Four controls bound the result. **K1** uses a random subspace of the same rank and shape. Our pre-registration also asked K1 to match the perturbation *magnitude* of the edit; the implementation matches rank only, so we log the deviation instead of hiding it. We measure the perturbation as the per-position ℓ_2 norm of the applied change, averaged over the positions of each prompt and then over 100 prompts and 12 targets, at prompt positions: 34.6 for the edit against 24.1 for K1, a ratio of 1.44, on activations of mean norm 169. We therefore add **K1^{norm}**, which uses the same random subspace but rescales the perturbation to the magnitude of the edit. **K2** is a sham edit that must reproduce the clean output token by token, and it does (100/100 on the selection prompts).

K3 borrows the subspace of another layer. **K4** transposes the prompt itself and gives the behavioral ceiling.

We score continuations with two measures. The *target-key rate* (TKR) is the fraction of continuations whose KS estimate equals the injected key, and we report it strictly. The *in-key rate* (IKR) is the fraction of continuation pitches that belong to a given scale, and it needs no estimator. We generate up to 384 new tokens per continuation and score its first 16 bars. Before we ran a single edit, we measured how far the perplexity of M-REF rises across 7,989 *natural* modulations in the validation data, and we froze the 90th percentile of that rise as our musicality budget, $\delta_{\text{PPL}}=0.613$. A continuation that exceeds the budget stays in the denominator and counts as a failure—we exclude nothing—and we call the resulting rate *guarded*. Every headline number below is guarded.

D. Selection phase and confirmatory phase

Our first sweep compared 8 layers and 3 subspace families on 100 prompts and evaluated the best cell (V-PROBE at L4) on those same prompts, which also lie inside the pool that trained the probe. Holm correction covered the 12 target keys, but it did not cover the selection of the layer and the subspace. We therefore treat that sweep as a *selection* phase, and we add a *confirmatory* phase. We draw 100 fresh prompts from a region of the test split that neither probe training nor any earlier run has touched. We fix the layer, subspace, timing, targets, controls, guard, metrics, tests, seeds, and sample size in a version-controlled document committed before the first confirmatory number exists. We repeat no layer or subspace search. All headline numbers come from the transposition-augmented model with seed 0; the second augmented seed replicates every selection-phase verdict. Because Sec. III-E shows what imbalanced target estimation can fabricate, we also verify the targets here: every major key contributes 7,530–9,237 positions to its class mean (max/min 1.23), and no mean is degenerate. We exclude identity cells, in which the injected key equals the key of the prompt, from the primary analysis, and we report them separately as a sanity check. **We froze all intervention choices and evaluation criteria before we evaluated the held-out set.**

III. RESULTS

A. The model computes the key instead of copying it from the surface

At L4 the probe reaches macro- F_1 0.927, the best pre-registered input baseline reaches 0.790, and the selectivity floor is 0.032, so the corrected margin is +0.105 (BCa 95% CI [0.091, 0.114]). This margin excludes zero in layers 1–7 of all six trained models, and DR-H1 holds in 6/6. It turns negative only at the raw embedding (−0.20), where the input surface should indeed win. An untrained model scores 0.243, so the architecture alone does not supply the key.

The probe attends the full prefix, while C3 saw at most 64 tokens, so history length alone might explain the margin. We therefore extended C3 to the full prefix and added two stronger variants: histograms with exponential recency decay, and a concatenation of a short and a long window. The window curve turns over inside the extension, at $W=96$ (0.801), and falls to 0.752 at $W=512$, because evidence from before a modulation poisons a histogram that cannot down-weight it. Against the strongest variant (0.824) the margin

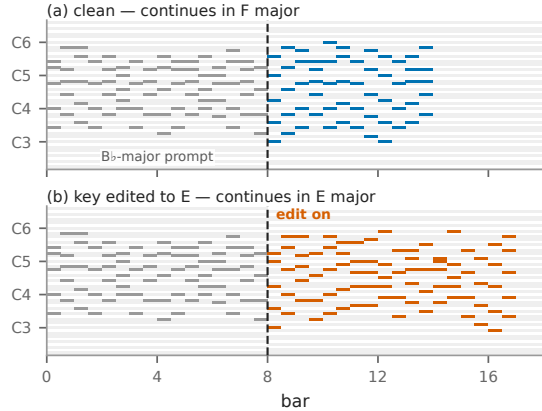


Fig. 1. The claim in one example (real output, fixed seed). We continue the same 8-bar Bb-major prompt (a) without an edit and (b) after we replace the key component of the layer-4 activations with the E-major class mean from bar 9 on. **The input does not change**, and the continuation follows the injected value. Shading marks the scale of the key in force.

stays positive at $+0.071$ $[0.056, 0.081]$, and all twelve size-and-seed verdicts stay unchanged. We therefore claim that the internal representation makes key more linearly accessible than these pitch-statistical baselines—not that it exceeds every function of the input, since the hidden state is itself one.

B. Editing the key representation changes the composed key

Selection phase. V-PROBE at L4 gave the best in-budget cell, with a guarded strict TKR of 0.378 against 0.075 for its layer-matched K1, a ratio of 5.0. All 12 targets separated after Holm correction ($p < 10^{-4}$), 87% of edits stayed in budget, and the edit reached 58% of the K4 ceiling (raw K4 TKR 0.652; the guard is undefined for K4, whose clean twin is untransposed, so the ratio divides the guarded edit by the raw ceiling—the least favorable convention). Pitch content agrees without the estimator: IKR is 0.937 for the injected key and 0.646 for the key of the prompt. The optimized subspace scored lower than the probe row space (V-DAS, 0.213), which matches the observation of Wu et al. [16] that a simple representation-based direction can outperform an optimized one. V-MEAN edits destroy the music at deep layers, where at most 5% stay in budget.

Confirmatory phase. The result holds on data that no earlier step touched. On the 100 held-out prompts, with every choice frozen beforehand and identity cells excluded (Fig. 2), the edit reaches a guarded TKR of 0.356 against 0.039 for K1, a ratio of 9.1. All 12 targets separate after Holm correction (worst $p = 4.8 \times 10^{-5}$; rank-biserial $r = 0.72$ – 1.00), and the paired prompt-level difference has a BCa 95% CI of $[0.284, 0.347]$. The guard admits 84% of edits, raw TKR is 0.410, and pitch content again crosses (IKR 0.936 for the injected key against 0.617 for the prompt’s). The edit also beats $K1^{\text{norm}}$, the control we rescale to the edit’s exact perturbation magnitude (0.056; 12/12 after Holm correction), and both random controls pass the guard more often than the edit does (0.96 and 0.93 against 0.84) while reaching only 0.041 and 0.059 unguarded: perturbation size does not explain the effect, and the guard costs the edit, not the

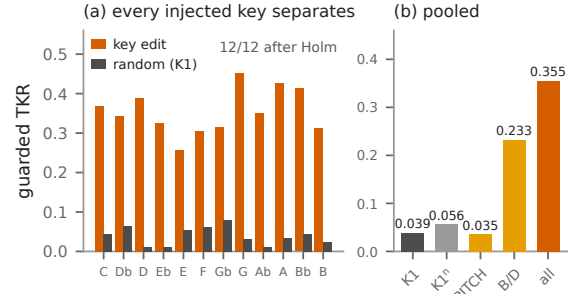


Fig. 2. The main causal result, on held-out prompts, with every choice frozen beforehand and identity cells excluded. (a) Guarded TKR for each injected key, edit against the rank-matched random control. (b) The same measure pooled, for the two random controls and the two restricted writes: K1 random; $K1^n$, the same random subspace rescaled to the edit’s perturbation magnitude; PITCH positions only; BAR/DUR positions only; and all positions.

controls. The held-out ceiling matches the selection phase: transposing the prompt itself (K4) reaches a raw TKR of 0.648, so the edit realizes 55% of full behavioral control. In the identity cells, where we inject the prompt’s own key, the model simply stays there (0.650).

Specificity. The edit does not merely leave the source key; it lands on the key we inject. On the held-out set, the estimated key of the continuation matches the injected key in 41.0% of cases, against 4.1% for K1, and only 8.5% of continuations stay in the key of the prompt, against 51.7% for K1. Most of the remaining mass falls on a neighbour a fifth away (34.8%), which the KS estimator is known to confuse. The edit therefore installs a specific counterfactual key rather than merely disrupting the current one.

C. The edit acts before the model hears a single new note

A sustained edit leaves one alternative explanation: the edit may perturb the first few notes, and the model may then re-estimate the key from its own output; we remove that explanation at the logit level. We extend each prompt with a teacher-forced, key-neutral structural prefix (BAR, POS_1), we apply the edit from the bar-9 boundary, and *we never sample a token*. We then read the next-token distribution over pitch tokens and score two frozen measures: P_{target} , the probability mass, under the full-vocabulary softmax, on pitch tokens of any octave whose pitch class is diatonic to the injected key (no renormalization over pitches), and $\Delta_{\text{key}} = \log P_{\text{target}} - \log P_{\text{source}}$.

All statistics are means of paired per-cell differences against the clean run, over non-identity prompt–target cells. The edit raises Δ_{key} by 0.695; the matched random edit, by 0.041. The mass on the target scale rises by 0.255 from a clean baseline of 0.547, against 0.017 for the control. The separation holds in 12/12 keys after Holm correction, with a worst-case $p = 5 \times 10^{-16}$ and rank-biserial $r \geq 0.99$. **The edit therefore changes the next musical decision of the model before any generated note can feed back.**

D. Where we can read the key is not where the model uses it

Two dissociations locate the effect. *Across layers*, the probe saturates over a broad band (macro- F_1 0.920–0.929 for L2–L6), but when we edit each layer in turn and subtract the

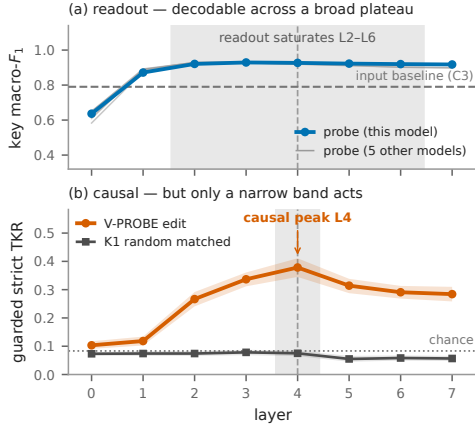


Fig. 3. (a) Probe macro- F_1 per layer for all six models (thin lines; seed 0 highlighted). We can read the key across a broad band. (b) Causal effect of the edit against the K1 matched control (bands: prompt-level bootstrap 95% CI). The edit changes the output only in a narrow band. Selection-phase data; the confirmatory run fixes L4.

matched control we obtain a single mid-network peak: 0.03, 0.04, 0.19, 0.26, **0.30**, 0.26, 0.23, 0.23 for L0–L7, and the best-against-median CI [0.038, 0.109] excludes zero. The edit does almost nothing at L0–L1, where the probe already reads the key well (Fig. 3). We claim middle-layer concentration rather than a landmark layer, because the second seed reproduces every verdict in the selection phase with its peak at L2. We did not obtain a held-out replication for that seed: its sham-edit gate failed on one prompt out of 100—floating-point non-associativity, not the intervention—and our frozen rule stops the run when the gate fails.

Across token types, we restrict the write by token type. An edit at PITCH positions alone does nothing (guarded TKR 0.077, at the K1 floor) and leaves the music intact, while an edit at BAR and DUR positions alone carries 69% of the full effect. The held-out run reproduces both halves: the PITCH-only arm reaches 0.036, below the K1 floor of 0.039, and separates from it in 0/12 targets, while the BAR/DUR arm reaches 0.233, or 65% of the full effect, and separates in 11/12. The PITCH-only arm does keep more continuations in budget (98% against 84%), but only because it barely changes the music. An intervention that simply steered pitch logits would show the opposite pattern. Together with the layer result, this shows that linear decodability is a weak guide to causal use, which parallels the feature-selection result of Arad et al. [17] in language models.

E. The result transfers to a public model trained on real music

We probe and edit the Anticipatory Music Transformer [28], trained on Lakh MIDI [29], MetaMIDI [30], and FMA [31]. We evaluate it on 300 Bach chorales [1], [32] with human local-key labels [33]. Here we read the probe where the model is about to choose a pitch, as the (time, duration, note) encoding requires; the supplementary shows the verdict inverts if we read after the note. The probe beats the surface baselines with a corrected margin of +0.196 [0.132, 0.260], while our own model of the same shape does not (+0.034, CI crosses zero). The two setups produce C3 baselines within 0.003 of each other, which confirms that we matched them.

This comparison varies the training data together with the tokenizer and training procedure, so it isolates the real-music training pipeline rather than the data alone.

The intervention also transfers. We re-froze the budget from natural modulations in the chorales and used a *different* public checkpoint as the judge. The edit then reaches $7.8\times$ the matched control (0.479 against 0.061), separates in 12/12 targets, keeps 99.7% of continuations in budget, and moves pitch content across (IKR 0.890 against 0.753). Its causal peak (L8) again differs from its probe peak (L10).

One first-run result did not survive its control, and we retract it. We appeared to find a key prior: the model never reached $F\sharp$ major, and success tracked how common each key is in the corpus (Spearman +0.91). We had estimated the edit targets from that same imbalanced corpus, however, and $F\sharp$ never occurs in it, so its target was a zero vector. Re-estimating from a key-balanced corpus lifts $F\sharp$ to 0.617 and drops the correlation to a non-significant +0.41. The failure mode transfers: edit targets from an imbalanced corpus can manufacture a false model prior.

F. What our results do not show

Three further claims, bounded with measurements (full analyses in the supplementary). *Capacity*: next-token accuracy stays flat from 0.5M to 86M parameters, yet the smallest model has no decodable key state and 90% of its edits leave the budget. *Persistence*: after a one-shot installation the pitch content stays shifted, but splicing the tokens of that one bar reproduces the entire effect: the model re-estimates the key at every step and holds no latent memory independent of the emitted notes. *Geometry*: the tonic directions form a circle of fifths, but the twelve transposition maps do not form a cyclic group (pre-registered H2b not supported).

IV. DISCUSSION AND LIMITATIONS

Our evidence supports a precise claim: within this training condition, the model computes an internal key representation and consults it during generation. It is manipulable, specific to the injected key, effective before feedback, and concentrated in middle layers and at structural token positions. Readers may call it a world model; for us the term is interpretation, not result. The gaps inform us as well: the edit reaches only 55–58% of the behavioral ceiling, and it fails exactly where the state is most readable (L0–L1, PITCH positions).

Limitations. We inject major keys only; we pre-registered minor and have not run it. The KS estimator confuses fifth-related keys; IKR triangulates that weakness. Our guard measures distributional damage under a reference model, not musicality; we ran no listening test. The public-model result covers one checkpoint at one size and keeps its original design (layer chosen on disjoint prompts, budget frozen before evaluation, no second held-out round).

V. CONCLUSION

A music Transformer does not merely expose correlates of key to a probe. With every choice fixed in advance, replacing a rank-24 component of the hidden state moves composition to the injected key—before any note can feed back, specifically toward the target, within a frozen budget, and in a public model trained on real music. Decodability and causal use come apart across layers and token positions; reading a representation does not show the model consults it. Testbed, freeze documents, and demos: <https://anonymous.4open.science/r/TBD>

REFERENCES

- [1] G. Hadjeres, F. Pachet, and F. Nielsen, “DeepBach: A steerable model for Bach chorales generation,” in *Proc. International Conference on Machine Learning*, 2017.
- [2] C.-Z. A. Huang *et al.*, “Music Transformer: Generating music with long-term structure,” in *Proc. International Conference on Learning Representations*, 2019.
- [3] J. Copet *et al.*, “Simple and controllable music generation,” in *Proc. Advances in Neural Information Processing Systems*, 2023.
- [4] M. Wei, M. Freeman, C. Donahue, and C. Sun, “Do music generation models encode music theory?” in *Proc. International Society for Music Information Retrieval Conference*, 2024.
- [5] J. Hewitt and P. Liang, “Designing and interpreting probes with control tasks,” in *Proc. Conference on Empirical Methods in Natural Language Processing*, 2019.
- [6] Y. Belinkov, “Probing classifiers: Promises, shortcomings, and advances,” *Computational Linguistics*, vol. 48, no. 1, pp. 207–219, 2022.
- [7] K. Li, A. K. Hopkins, D. Bau, F. Viégas, H. Pfister, and M. Wattenberg, “Emergent world representations: Exploring a sequence model trained on a synthetic task,” in *Proc. International Conference on Learning Representations*, 2023.
- [8] N. Nanda, A. Lee, and M. Wattenberg, “Emergent linear representations in world models of self-supervised sequence models,” in *Proc. BlackboxNLP Workshop on Analyzing and Interpreting Neural Networks for NLP*, 2023.
- [9] A. Karvonen, “Emergent world models and latent variable estimation in chess-playing language models,” in *Proc. Conference on Language Modeling*, 2024.
- [10] A. F. Spies, W. Edwards, M. I. Ivanitskiy, A. Skapars, T. Räuker, K. Inoue, A. Russo, and M. Shanahan, “Transformers use causal world models in maze-solving tasks,” *arXiv preprint arXiv:2412.11867*, 2024.
- [11] X. Zhang, “What do world models learn in RL? probing latent representations in learned environment simulators,” *arXiv preprint arXiv:2603.21546*, 2026.
- [12] K. Li, O. Patel, F. Viégas, H. Pfister, and M. Wattenberg, “Inference-time intervention: Eliciting truthful answers from a language model,” in *Proc. Advances in Neural Information Processing Systems*, 2023.
- [13] A. Zou *et al.*, “Representation engineering: A top-down approach to AI transparency,” *arXiv preprint arXiv:2310.01405*, 2023.
- [14] J. Koo, G. Wichern, F. G. Germain, S. Khurana, and J. Le Roux, “SMITIN: Self-monitored inference-time intervention for generative music transformers,” *IEEE Open Journal of Signal Processing*, vol. 6, pp. 266–275, 2025.
- [15] D. Zhao, D. Beaglehole, T. Berg-Kirkpatrick, J. McAuley, and Z. Novack, “Steering autoregressive music generation with recursive feature machines,” in *Proc. International Conference on Learning Representations*, 2026.
- [16] Z. Wu, A. Arora, A. Geiger, Z. Wang, J. Huang, D. Jurafsky, C. D. Manning, and C. Potts, “AxBench: Steering LLMs? even simple baselines outperform sparse autoencoders,” in *Proc. International Conference on Machine Learning*, 2025, pp. 67 035–67 080.
- [17] D. Arad, A. Mueller, and Y. Belinkov, “SAEs are good for steering—if you select the right features,” in *Proc. Conference on Empirical Methods in Natural Language Processing*, 2025.
- [18] C. L. Krumhansl and E. J. Kessler, “Tracing the dynamic changes in perceived tonal organization in a spatial representation of musical keys,” *Psychological Review*, vol. 89, no. 4, pp. 334–368, 1982.
- [19] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Proc. Advances in Neural Information Processing Systems*, 2017.
- [20] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, “Language models are unsupervised multitask learners,” *OpenAI Technical Report*, 2019.
- [21] G. Alain and Y. Bengio, “Understanding intermediate layers using linear classifier probes,” in *Proc. International Conference on Learning Representations, Workshop Track*, 2017.
- [22] C. L. Krumhansl, *Cognitive Foundations of Musical Pitch*. Oxford University Press, 1990.
- [23] D. Temperley, “What’s the key for a piece of music? The Krumhansl–Schmuckler key-finding algorithm reconsidered,” *Music Perception*, vol. 17, no. 1, pp. 65–100, 1999.
- [24] B. Efron, “Better bootstrap confidence intervals,” *Journal of the American Statistical Association*, vol. 82, no. 397, pp. 171–185, 1987.
- [25] A. Geiger, H. Lu, T. Icard, and C. Potts, “Causal abstractions of neural networks,” in *Proc. Advances in Neural Information Processing Systems*, 2021.
- [26] K. Meng, D. Bau, A. Andonian, and Y. Belinkov, “Locating and editing factual associations in GPT,” in *Proc. Advances in Neural Information Processing Systems*, 2022.
- [27] A. Geiger, Z. Wu, C. Potts, T. Icard, and N. D. Goodman, “Finding alignments between interpretable causal variables and distributed neural representations,” in *Proc. Conference on Causal Learning and Reasoning*, 2024.
- [28] J. Thickstun, D. Hall, C. Donahue, and P. Liang, “Anticipatory music transformer,” *Transactions on Machine Learning Research*, 2024.
- [29] C. Raffel, “Learning-based methods for comparing sequences, with applications to audio-to-MIDI alignment and matching,” Ph.D. dissertation, Columbia University, 2016.
- [30] J. Ens and P. Pasquier, “Building the MetaMIDI dataset: Linking symbolic and audio musical data,” in *Proc. International Society for Music Information Retrieval Conference*, 2021.
- [31] M. Defferrard, K. Benzi, P. Vandergheynst, and X. Bresson, “FMA: A dataset for music analysis,” in *Proc. International Society for Music Information Retrieval Conference*, 2017.
- [32] C.-Z. A. Huang, T. Cozijmans, A. Roberts, A. Courville, and D. Eck, “Counterpoint by convolution,” in *Proc. International Society for Music Information Retrieval Conference*, 2017.
- [33] M. Gotham, G. Micchi, N. N. López, and M. Sailor, “When in Rome: A meta-corpus of functional harmony,” *Transactions of the International Society for Music Information Retrieval*, 2023.